

JSON-Based Sequencer and Customizable Assets Specification Proposal

Overview

This specification defines how the `sequence.json` drives the immersive experience, managing localized audio, visuals, and text instructions, while keeping static UI assets (like the panel background and welcome screen graphics) in a separate, clearly structured folder for allowing the customization of the visual appearance of the app.

The **Sequencer**, as defined by the `sequence.json` file, must be capable of the following:

- Sending **Arduino commands**
- Displaying and hiding **images or videos** (one at a time only, with preferred resolution)
- Showing/hiding **UI panels and instructional text**
- Playing localized **audio** in **4 languages** (de, en, fr, it)

On the other hand, the following **Static assets** can be defined :

- Panel **background** image
- **Start button** image
- **Language buttons** images
- The **font** used in UI panels

Sequence Step Format

Each timeline event is represented as a step in the JSON. The format for each step is as follows:

```
{
  "time": 0.0,
  "textKey": "Welcome",
  "audio": "welcome.mp3",
  "image": "instructions.png"
  "actions": []
}
```

Field Descriptions:

- **time**: Time (in seconds) since the experience starts (triggered when both users are ready).
- **textKey**: A key that maps to a localized instruction string. Also shows the text panel if not already showing.
- **audio**: File name of the audio instruction. Separate files must exist for each language in their respective folders.
- **visual**: Image (.png, .jpg) or video (.mp4) — displayed full-screen
- **actions**: List of actions to perform at this step.

Supported Actions

You can include one or more of the following actions in any step:

- ShowVisual
- HideVisual
- HidePanel
- WallOn
- WallOff
- MirrorOn
- MirrorOff
- StartExperience
- EndExperience

Language Support

- Language selection uses country codes: `en`, `de`, `fr`, `it`, etc.
- Audio stored in `/Sequence/Audio/{lang}/`
- Translation CSVs in `/Sequence/Translations/{lang}.csv`
- Flags in `/Static/` using `flag_{code}.png`

Folder Structure

Make sure the following folder structure is followed to ensure all the assets are loaded correctly :

```

/TheMachineToBeAnother/
├─ TheMachineToBeAnother.exe
├─ /Content/
│   └─ /Sequence/
│       ├── sequence.json
│       ├── Audio/
│       │   ├── en/
│       │   │   └─ welcome.mp3
│       │   ├── de/
│       │   │   └─ welcome.mp3
│       │   ├── fr/
│       │   │   └─ welcome.mp3
│       │   └─ it/
│       │       └─ welcome.mp3
│       ├── Images/
│       │   └─ instructions.png
│       ├── Videos/
│       │   └─ instructions.mp4
│       └─ Translations/
│           ├── en.csv
│           ├── de.csv
│           ├── fr.csv
│           └─ it.csv
└─ /Static/
    ├── panel_background.png
    ├── start_button.png
    └─ flag_en.png
  
```

```
|      └─ flag_de.png
|      └─ flag_fr.png
|      └─ flag_it.png
|      └─ font.ttf
```

Enclosed is a working template `sequence.json` file that replicates the current behavior of the Unity Timeline. This can be used as a reference or starting point for customization.

Notes on Implementation

- The software is already being prepared to support this format.
- I have not begun implementation of the JSON sequence runner itself, pending your confirmation.
- Once the specification is agreed upon, development and integration should proceed quickly.
- There will not be a user-friendly UI for timeline editing at this stage. Adjusting timings will require manually editing the JSON.

Optional Debug Features

To simplify timeline adjustment during testing, we can consider adding, if there is still time:

- Key bindings to move forward/backward between steps
- A short on-screen display of the currently active step's JSON for debugging

Let me know what you think and if I should move forward